Stats 102B Homework 2

Christian Jordan

2026-04-27

(1) Two-Dimensional Random Search

(a) random_search()

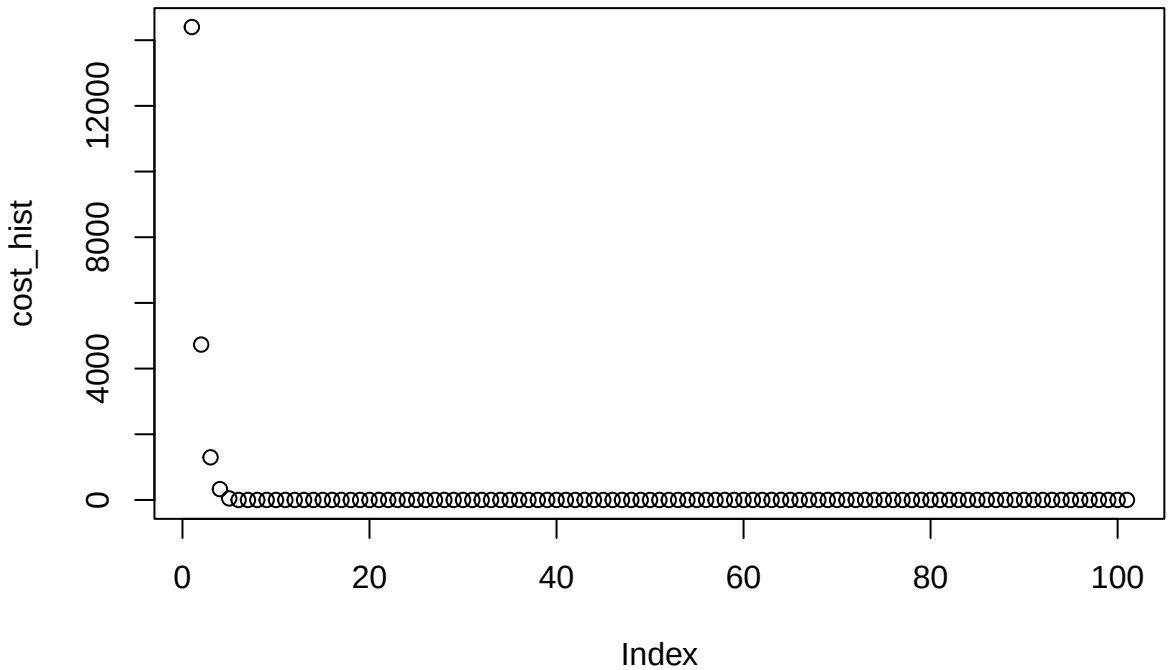
```
random_search <- function(g, alpha_choice = 1, max_its = 100, num_samples = 1000, w0 = c(0,0)) {
  # Initialize weight and cost histories
  weight_history <- matrix(w0, nrow = 2)
  cost_history <- g(w0[1], w0[2])

  its <- 1
  while(its <= max_its) {
    current_weights <- weight_history[,its]
    weights <- matrix(numeric(0), nrow = 2)
    costs <- numeric(0)

    # Generate random directions
    for(i in 1:num_samples) {
      direction <- runif(min = -1, max = 1, n = 2)
      direction <- direction / sqrt(sum(direction^2))

      # Save random weight update and cost
      if(alpha_choice != "diminishing") {
        weights <- cbind(weights, current_weights + alpha_choice * direction)
        costs <- c(costs, g(weights[1, i], weights[2, i]))
      } else {
        weights <- cbind(weights, current_weights + (1/its) * direction)
        costs <- c(costs, g(weights[1, i], weights[2, i]))
      }
    }
    # Save overall best weights and cost found
    best_index <- which.min(costs)
    best_weight <- weights[, best_index]
    best_cost <- costs[best_index]
    weight_history <- cbind(weight_history, unname(best_weight))
    cost_history <- c(cost_history, best_cost)

    its <- its + 1
  }
  list(weight_history, cost_history)
}
```

Suppose we have $g(w_1, w_2 = 100(w_2 - w_1^2)^2 + (w_1 - w_2)^2$

```
g <- function(w1, w2) {
  100 * (w2 - w1^2)^2 + (w1 - w2)^2
}
```

(b) Draw the plots of cost_history to showcase your function with $w_0 = [-3 - 3]^T$**.**

```
cost_hist <- random_search(g, w0 = c(-3, -3))[[2]]
plot(cost_hist)
```

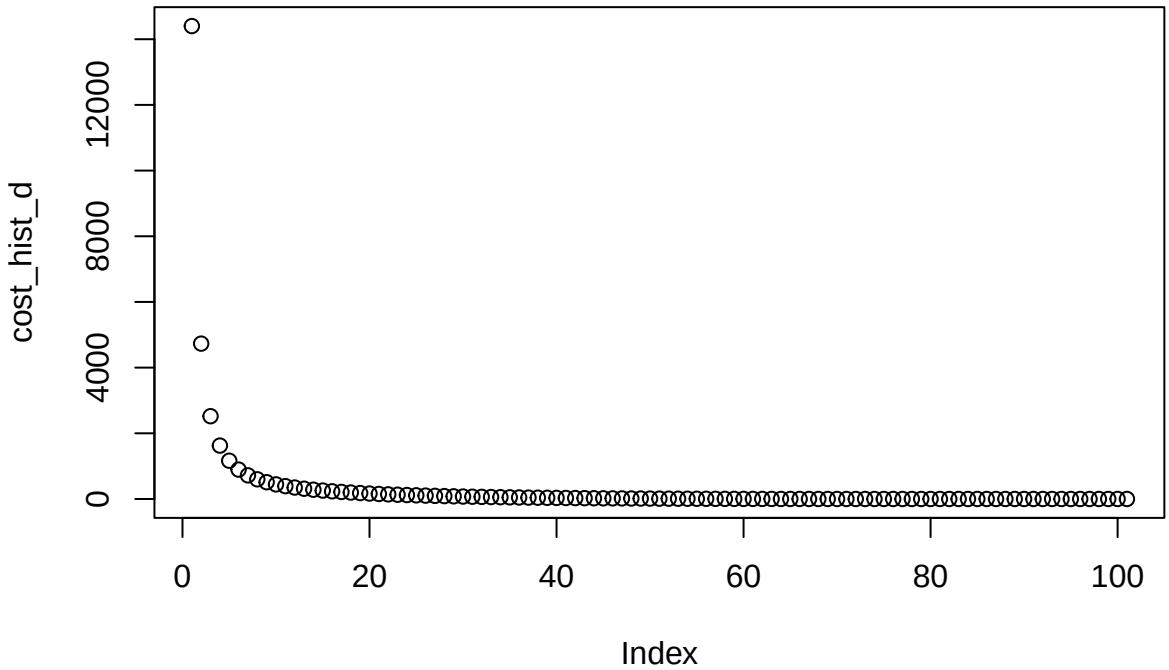



```
c(Minima = tail(cost_hist, n = 1))
```

```
##      Minima
## 0.002401151
```

(c) Repeat (b) with a steplength diminishing rule.

```
cost_hist_d <- random_search(g, w0 = c(-3, -3), alpha_choice = "diminishing")[[2]]
plot(cost_hist_d)
```




```
c(Minima = tail(cost_hist_d, n = 1))
```

```
##       Minima
## 5.843231e-08
```

(d) Discuss your results of (b) and (c)

From the plots, we can see that the fixed step length method used in part (b) converges to the estimated optimum sooner (less than 10 iterations) compared to the diminishing-step method used in part (c) which doesn't converge to a final value until after around 60 iterations. However, we can from the code below that the fixed step length method settled at weights resulting in a cost function value of 0.016 while the diminishing step length method settled at weights resulting in a value of 1.01*10^-16. So while the diminishing method took longer to converge, it converged to a better minima for the function. This is because it is less likely to overshoot the actual minima of the function as it gets closer and closer to it more slowly.

```
cost_hist[length(cost_hist)]
```

```
## [1] 0.002401151
```

```
cost_hist_d[length(cost_hist_d)]
```

```
## [1] 5.843231e-08
```

(2) Zero-order Coordinate Search

(a) Divise pseuocode and implement zero-order coordinate descent for 2D input

```
# Pseudocode:
# Function {
# Initialize weight and cost histories
#
# For iterations 1 to max_its or until tol reached {
#   If odd iteration -> optimize w1
#       Calculate step in + and - w1 directions
#       Save the direction that produces best descent (or neither if no descent)
#       Update cost weight and cost history
#   else, optimize w2
#       Calculate step in + and - w2 directions
#       Save the direction that produces best descent (or neither if no descent)
#       Update cost weight and cost history
# }
#
# Return final weight and cost
# }
```

```
coordinate_descent_0 <- function(g, alpha_choice = 1, max_its = 100, w0 = c(0,0), tol = 10^-5) {
  # Initialize weight and histories
  weight_history <- matrix(w0, nrow = 2, ncol = 1)
  cost_history <- g(w0[1], w0[2])

  its <- 1
  no_improvement <- 0

  while(its <= max_its) {
    current_weights <- unname(weight_history[,its])
    current_cost <- cost_history[its]

    # Exit if tolerance is met
    if(its > 2 && abs(current_cost - cost_history[its-1]) <= tol && no_improvement >= 2) {
      break
    }

    if(alpha_choice == "diminishing") {
      d <- 1/its
    } else {
      d <- alpha_choice
    }

    # Alternate coordinate to descend
    if(its %% 2 == 1) {
      weight_plus <- current_weights + c(d,0)
      weight_minus <- current_weights - c(d,0)
    } else {
      weight_plus <- current_weights + c(0, d)
      weight_minus <- current_weights - c(0, d)
```

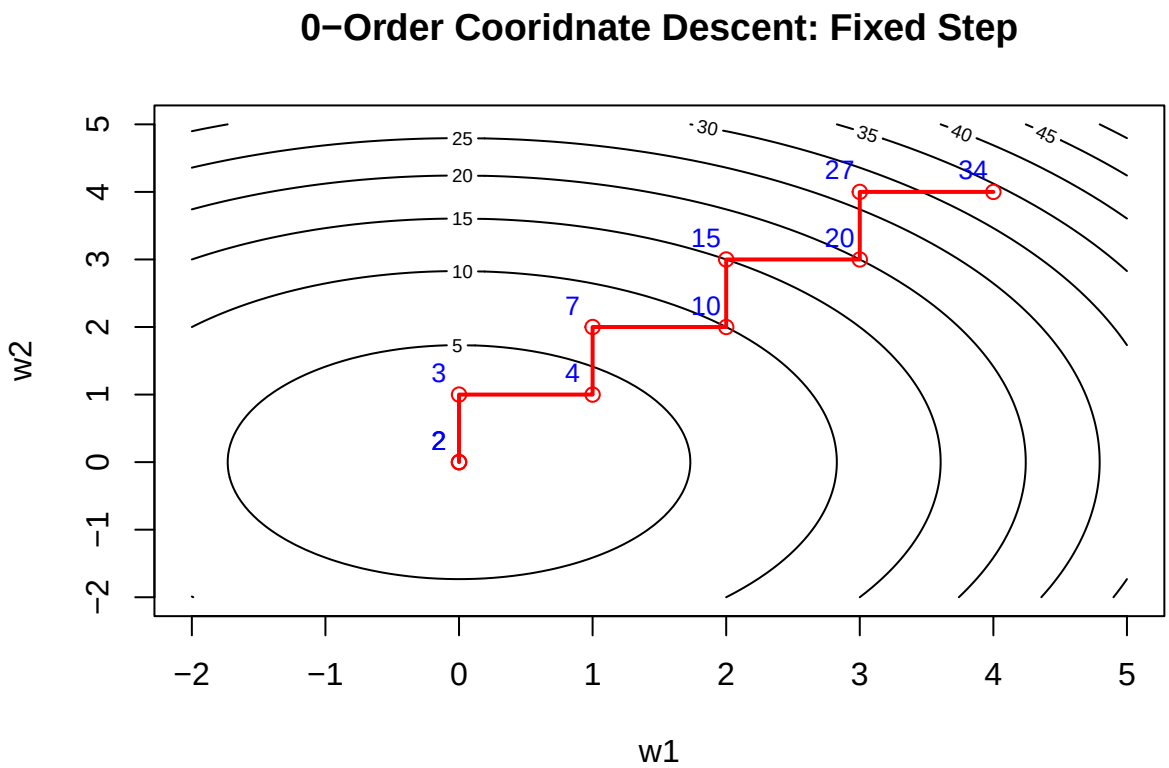
```r
    }

    cost_plus <- g(weight_plus[1], weight_plus[2])
    cost_minus <- g(weight_minus[1], weight_minus[2])

    if(current_cost <= cost_plus && current_cost <= cost_minus) { # Other options are worse
      weight_history <- cbind(weight_history, unname(current_weights))
      cost_history <- c(cost_history, current_cost)
      no_improvement <- no_improvement + 1
    } else {
      if(cost_plus <= cost_minus) { # + direction is better
        weight_history <- cbind(weight_history, unname(weight_plus))
        cost_history <- c(cost_history, cost_plus)
        no_improvement <- 0
      } else { # - direction is better
        weight_history <- cbind(weight_history, unname(weight_minus))
        cost_history <- c(cost_history, cost_minus)
        no_improvement <- 0
      }
    }

    its <- its + 1
  }

  list(weight_history, cost_history)
}
```

Suppose we have $g(w_1, w_2 = w_1^2 + w_2^2 + 2$

```
g2 <- function(w1, w2) {
  w1^2 + w2^2 + 2
}
```

(b) Draw the plot of the cost function against weight history and the cost history plot to showcase your function with $w_0 = [44]^T$.

```
# Minimize function
results <- coordinate_descent_0(g2, w0=c(4,4))
weights <- results[1]
costs <- results[2]

# Contour plot
w1_vals <- seq(-2, 5, length.out = 100)
w2_vals <- seq(-2, 5, length.out = 100)
z <- outer(w1_vals, w2_vals, g2)

contour(w1_vals, w2_vals, z,
        xlab = "w1",
        ylab = "w2")

# Plot weight values
points(weights[[1]][1,], weights[[1]][2,], col = "red")

# Connect weight values
lines(weights[[1]][1,], weights[[1]][2,],
      col = "red", lwd = 2)

# Plot cost label
text(weights[[1]][1,] - 0.15, weights[[1]][2,]-0.1,
     labels = costs[[1]],
     pos = 3,
     cex = 0.8,
     col = "blue")

title("0-Order Cooridnate Descent: Fixed Step")
```

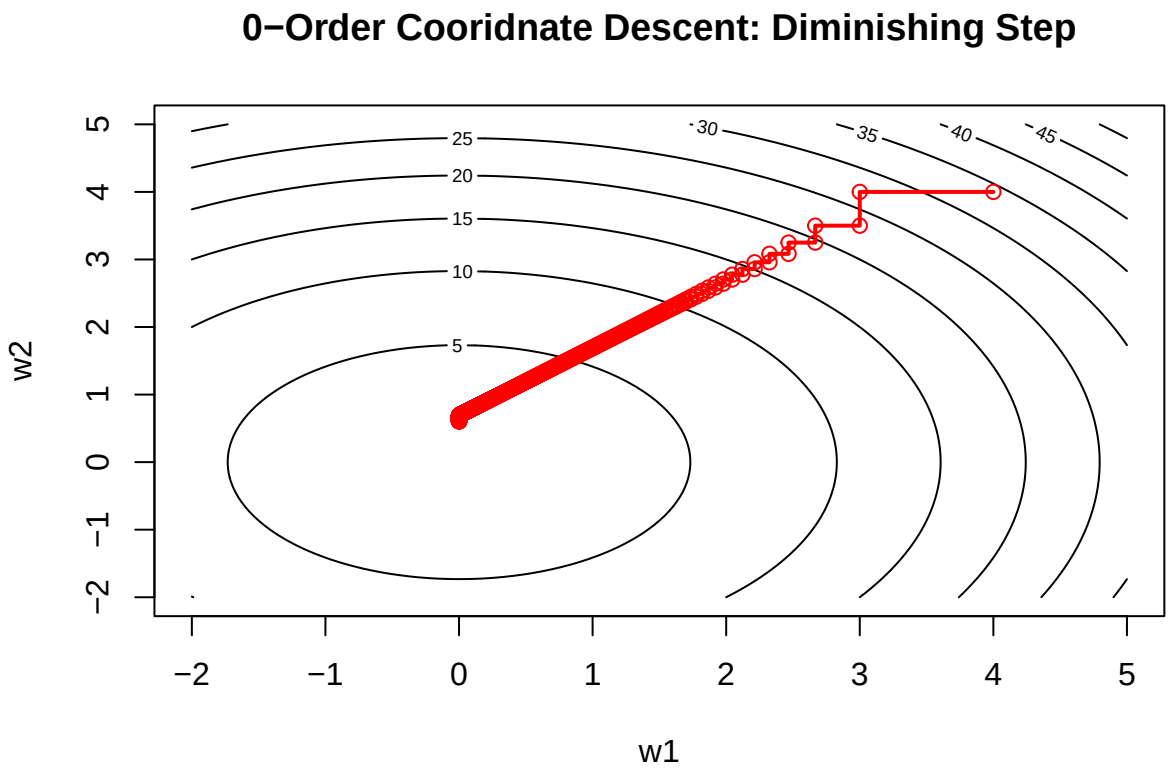



```r
c(Minima = tail(unlist(costs), n = 1))
```

```
## Minima
##      2
```

(c) Repeat (b) with diminishing steplength

```r
# Minimize function
results <- coordinate_descent_0(g2, w0=c(4,4), alpha_choice = "diminishing", max_its = 1000)

weights <- results[1]
costs <- results[2]

# Contour plot
w1_vals <- seq(-2, 5, length.out = 100)
w2_vals <- seq(-2, 5, length.out = 100)
z <- outer(w1_vals, w2_vals, g2)

contour(w1_vals, w2_vals, z,
        xlab = "w1",
        ylab = "w2")

# Plot weight values
points(weights[[1]][1,], weights[[1]][2,], col = "red")

# Connect weight values
lines(weights[[1]][1,], weights[[1]][2,],
      col = "red", lwd = 2)

# Plot cost label
# text(weights[[1]][1,] - 0.15, weights[[1]][2,]-0.1,
#      labels = round(costs[[1]], digits = 2),
#      pos = 3,
#      cex = 0.8,
#      col = "blue")

title("0-Order Cooridnate Descent: Diminishing Step")
```




```r
c(Minima = tail(unlist(costs), n = 1))
```

```
##   Minima
## 2.364319
```

Given $g(w_1, w_2 = 0.26(w_1^2 + w_2^2) - 0.48 w_1 w_2$

```
g3 <- function(w1, w2) {
  0.26 * (w1^2 + w2^2) - 0.48 * w1 * w2
}
```

(d) Find the global minimum and explain the way you do.

```
set.seed(1)

# Minimize the function
results <- list()
for (i in 1:100) {
  coord <- runif(2, min = -5, max = 5)
  results[[i]] <- coordinate_descent_0(g3, w0=coord, alpha_choice = "diminishing", tol = 1e-8)
}

best_weights <- integer(0)
best_results <- integer(0)

for(i in 1:100) {
  weights_i <- results[[i]][[1]]
  costs_i   <- results[[i]][[2]]

  best_weights <- cbind(best_weights, weights_i[, ncol(weights_i)])
  best_results <- c(best_results, tail(costs_i, 1))
}

minimum <- which(best_results == min(best_results))
best_weights[,minimum]
```

```
## [1] -0.001666165  0.003896475
```

```
best_results[minimum]
```

```
## [1] 7.785484e-06
```

The global minimum of the cost function from the function is 7.785-06, with weights of w1 = -0.002 and w2 = 0.003. This is approximately a value of 0 at weights w1 = 0 and w2 = 0.

I minimize the function globally using coordinate descent with diminishing step length at 100 random weight values ranging from -5 to 5. By doing this, the function is more likely to find a global minimum in that search area rather than finding a local minimum.

(3) First-order Coordinate Descent

(a) Express coordinate descent as a local optimization scheme

First order coordinate descent is a local optimization scheme because at each step, it is optimizing along only one coordinate axis. Rather than attempting to find the minimum of the entire funtion, it only considers one axes while holding the other(s) constant, allowing for many local optimization steps that eventually lead to global or local optimization of the whole function.

```
# Pseudocode
#
# Set k = 0
#
# while ( > max iterations or ||w^(k+1) - w^(k)|| < tol)
#   For i = 1 to d {
#        // minimize along coordinate i
#         w_i^(k+1) = argmin_z f(w_1^(k+1), ..., w_{i-1}^(k+1), z, w_{i+1}^(k), ..., w_d^(k))
#   }
#
#    Set k = k + 1
# }
#
# Output: w^(k)
```

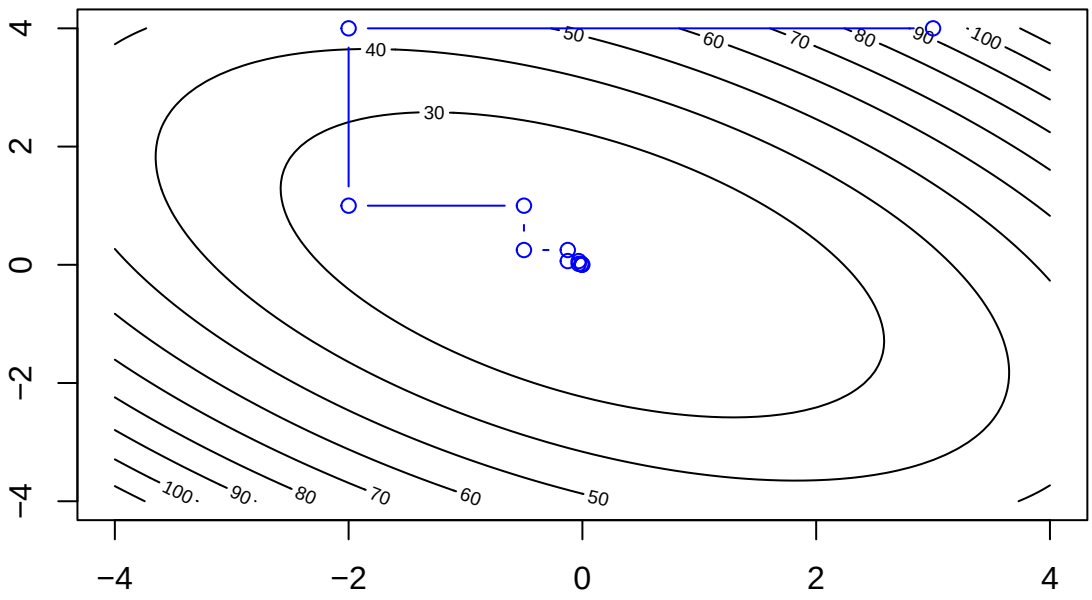
Given $g(w_1, w_2 = 2w_1^2 + 2w_2^2 + 2w_1w_2 + 20$

```
g4 <- function(w1, w2) {
  2 * w1^2 + 2 * w2^2 + 2 * w1 * w2 + 20
}
```

(b) Write code for the particular case of Example 2 from Chapter 3 and repeat the experiment

```
w0 <- c(3, 4)
w <- matrix(w0, nrow = 1)

tol <- 10^-4
max_its <- 100

i <- 1

repeat{
  for(j in 1:2){

    i <- i + 1
    w <- rbind(w, w[i-1,])

    if(j == 1){
      # minimize w1 given w2
      w[i,1] <- -0.5 * w[i,2]
    } else {
      # minimize w2 given w1
      w[i,2] <- -0.5 * w[i,1]
    }
  }

  if(i > max_its || abs(g4(w[i,1], w[i,2]) - g4(w[i-1,1], w[i-1,2])) < tol) {
    break
  }
}

xgrid <- ygrid <- seq(-4, 4, 0.1)
g_out <- outer(xgrid, ygrid, FUN = g4)

contour(x = xgrid, y = ygrid, z = g_out)

points(w[,1], w[,2], type = "b", col = "blue")

title("1st Order Cooridnate Descent: Fixed Step")
```

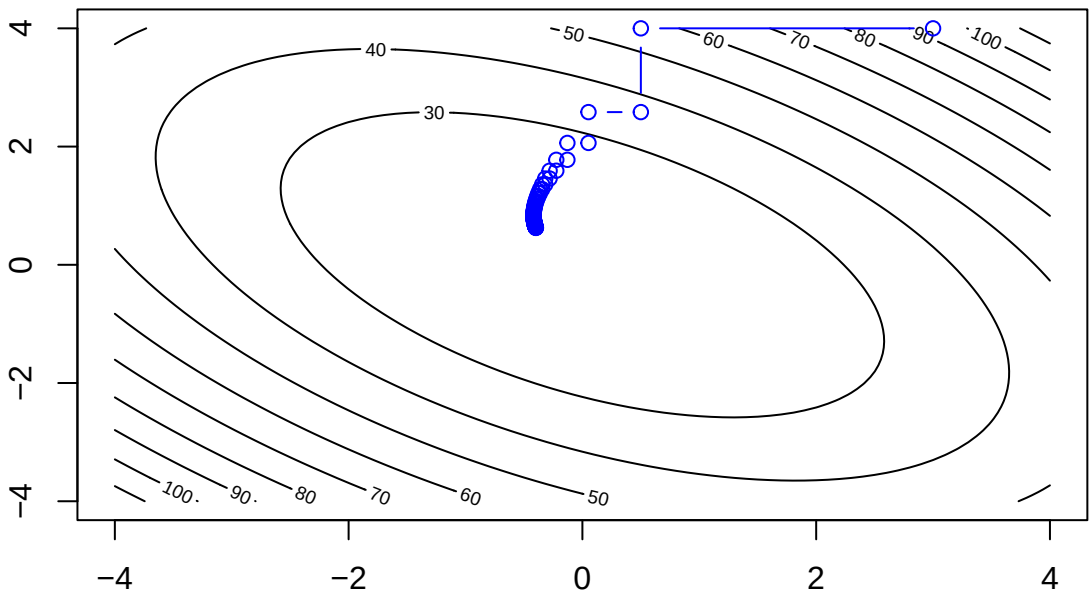
1st Order Cooridnate Descent: Fixed Step



(c) Repeat (b) with a diminishing steplength

```
w0 <- c(3, 4)
w <- matrix(w0, nrow = 1)

tol <- 10^-4
max_its <- 100

i <- 1

repeat{
  for(j in 1:2){

    i <- i + 1
    w <- rbind(w, w[i-1,])

    d <- 1 / i

    if (j == 1) {
      w[i,1] <- w[i-1,1] + d * ( -0.5 * w[i,2] - w[i-1,1])
    } else {
      w[i,2] <- w[i-1,2] + d * (-0.5 * w[i,1] - w[i-1,2])
    }
  }

  if(i > max_its || abs(g4(w[i,1], w[i,2]) - g4(w[i-1,1], w[i-1,2])) < tol) {
    break
  }
}

xgrid <- ygrid <- seq(-4, 4, 0.1)
g_out <- outer(xgrid, ygrid, FUN = g4)

contour(x = xgrid, y = ygrid, z = g_out)

points(w[,1], w[,2], type = "b", col = "blue")

title("1st Order Cooridnate Descent: Diminishing Step")
```

1st Order Cooridnate Descent: Diminishing Step



(4) Convex Function Properties

Prove that:

(1) The sum of convex functions is always convex

Given two convex functions $f(x)$ and $g(x)$:

$$f[\lambda x_1 + (1-\lambda)x_2] \leq \lambda f(x_1) + (1-\lambda)f(x_2)$$
$$g[\lambda x_1 + (1-\lambda)x_2] \leq \lambda g(x_1) + (1-\lambda)g(x_2)$$

If we add these, then:

$$f[\lambda x_1 + (1-\lambda)x_2] + g[\lambda x_1 + (1-\lambda)x_2] \leq \lambda f(x_1) + (1-\lambda)f(x_2) + \lambda g(x_1) + (1-\lambda)g(x_2)$$

$$f[\lambda x_1 + (1-\lambda)x_2] + g[\lambda x_1 + (1-\lambda)x_2] \leq \lambda[f(x_1) + g(x_1)] + (1-\lambda)[f(x_2) + g(x_2)]$$

If we define $h(x) = f(x) + g(x)$, then we can write as:

$$h[\lambda x_1 + (1-\lambda)x_2] \leq \lambda h(x_1) + (1-\lambda)h(x_2)$$

This is the same definition of a convex function. Therefore, because $h(x)$ is convex and $h(x) = f(x) + g(x)$ which are also convex functions, the sum of any two convex functions is also convex. This can also be extended to summation of more than 2 convex functions, the result will still be convex.

(2) The maximum of convex functions is convex

Let $h(x)$ be the max of two convex functions $f(x)$ and $g(x)$:

$$h(x) = \max(f(x), g(x))$$

Substitute in for x:

$$h(\lambda x_1 + (1-\lambda)x_2) = \max\{f(\lambda x_1 + (1-\lambda)x_2), g(\lambda x_1 + (1-\lambda)x_2)\}$$

$$\leq \max\{\lambda f(x_1) + (1-\lambda)f(x_2), \lambda g(x_1) + (1-\lambda)g(x_2)\}$$

$$\leq \lambda \max\{f(x_1), g(x_1)\} + (1-\lambda)\max\{f(x_2), g(x_2)\} = \lambda h(x_1) + (1-\lambda)h(x_2)$$

So, we have that:

$$h(\lambda x_1 + (1-\lambda)x_2) \leq \lambda h(x_1) + (1-\lambda)h(x_2)$$

This is the definition of a convex function, meaning that $h(x)$ is convex. Because $h(x) = max(f(x), g(x))$ and they are convex functions, that means that the maximum of convex functions is also convex.

(5) Gradient Descent

(a) Divise pseudocode and implement a gradient descent function

```
# Pseudocode:
#
# weight history <- w0
# cost history <- g(w0)
#
# i <- 1
#
# while i < max_its {
#
#  if diminshing, alpha = 1/k
#  else, alpha = alpha_choice
#
#  gradient = g_prime(current weights)
#
#  new weights = current weights - alpha * gradient
#
#  weight history adds new weights
#  cost history adds g(new weights)
#
#  if tol met, break loop
# }
#
# Return list of weight and cost histories
```

```
gd <- function(g, g_prime, alpha_choice = 1, max_its = 100, w0, tol = 10^-5) {

  # Initialize
  weight_history <- matrix(w0, nrow = length(w0), ncol = 1)
  cost_history <- g(w0)

  its <- 1

  while (its < max_its) {

    # Current weights
    w <- weight_history[, ncol(weight_history)]

    # Step size
    if (alpha_choice == "diminishing") {
      alpha <- 1 / its
    } else {
      alpha <- alpha_choice
    }

    # Gradient
    gradient <- g_prime(w)
```

```r
    # Update
    w_new <- w - alpha * gradient

    # Store
    weight_history <- cbind(weight_history, unname(w_new))
    cost_history <- c(cost_history, g(w_new))

    # Convergence
    if (sqrt(sum((w_new - w)^2)) <= tol) {
      break
    }

    its <- its + 1
  }

  list("weight_history" = weight_history, "cost_history" = cost_history)
}
```

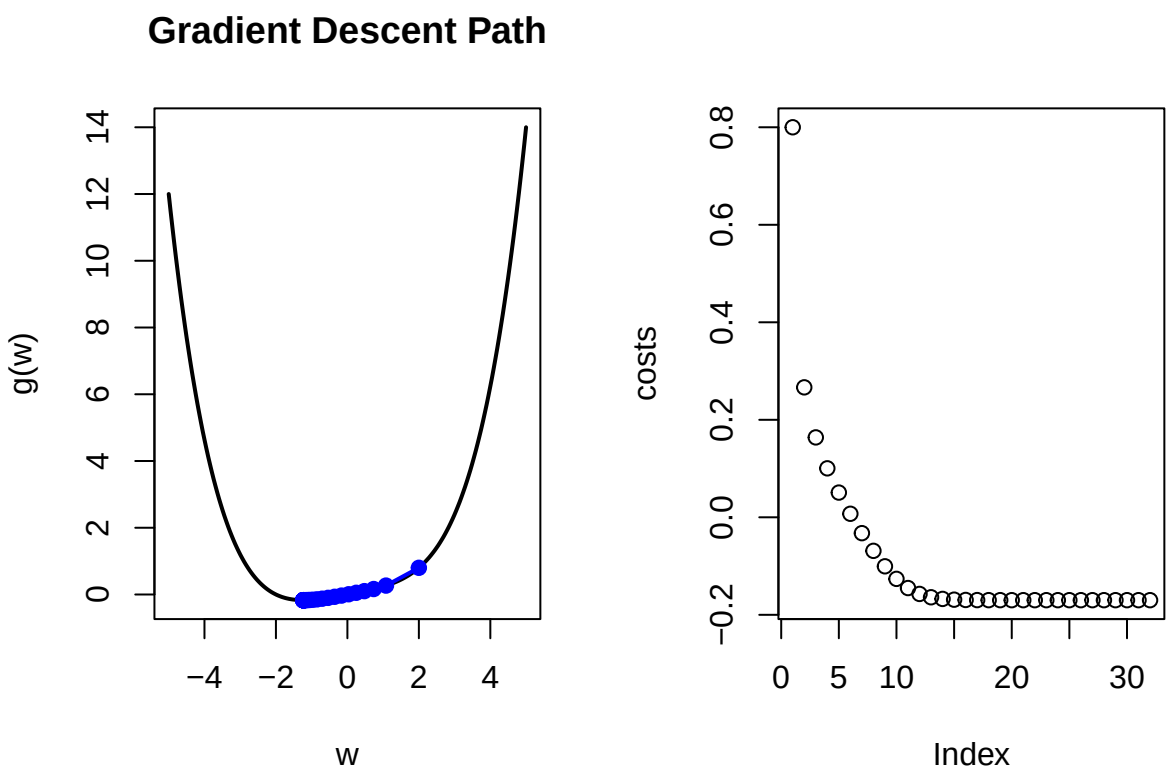
Given the function $g(w) = \frac{1}{50}(w^4 + w^2 + 10w)$:

```r
g5 <- function(w) {
  1/50 * (w^4 + w^2 + 10*w)
}

gprime <- function(w) {
  1/50 * (4*w^3 + 2*w + 10)
}
```

```r
plot_gd <- function(results) {
  weights <- as.numeric(results$weight_history)
  costs <- results$cost_history

  x_vals <- seq(-5, 5, length.out = 200)
  y_vals <- g5(x_vals)

  par(mfrow = c(1, 2))

  plot(x_vals, y_vals, type = "l", lwd = 2,
      xlab = "w", ylab = "g(w)",
      main = "Gradient Descent Path")

  points(weights, costs, col = "blue", pch = 19)
  lines(weights, costs, col = "blue", lwd = 2)

  plot(costs)

  costs[length(costs)]
}
```

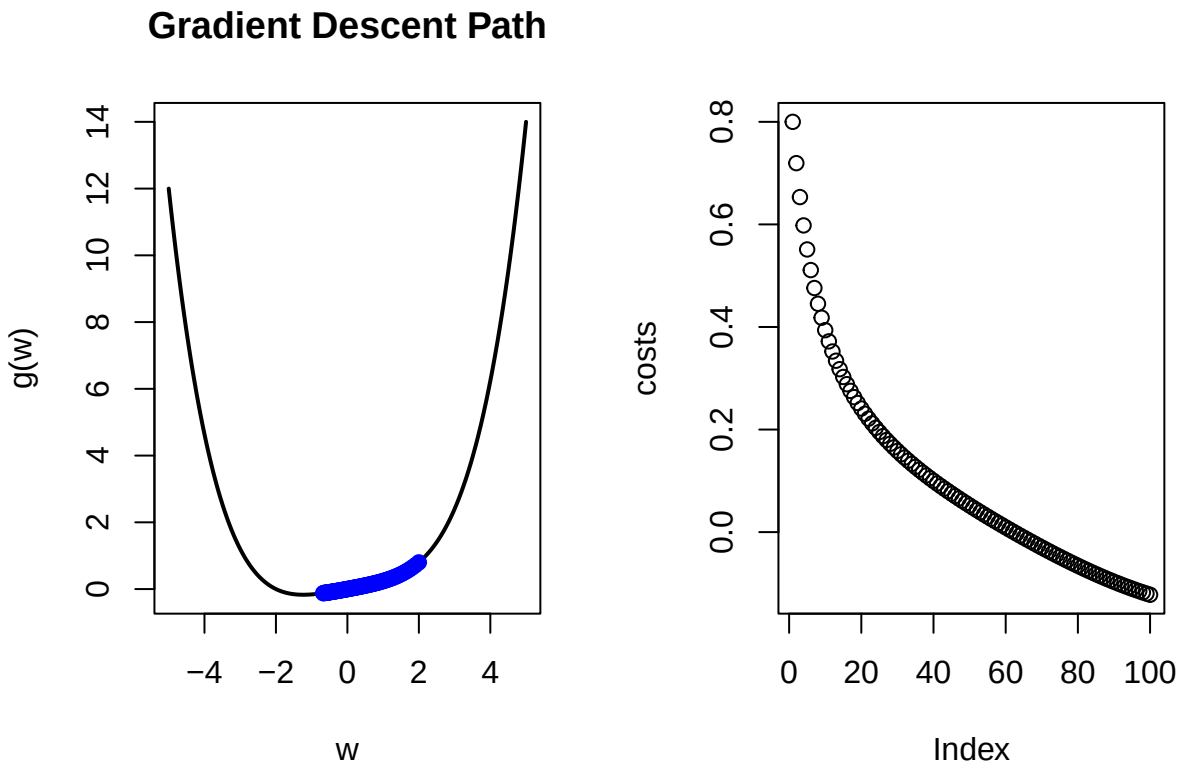
(b) Make three runs with alpha = 1, 0.1, and 0.01 and plot them (use initial point w = 2). Discuss your results

```
results <- gd(g5, gprime, alpha_choice = 1, w0 = 2)
plot_gd(results)
```



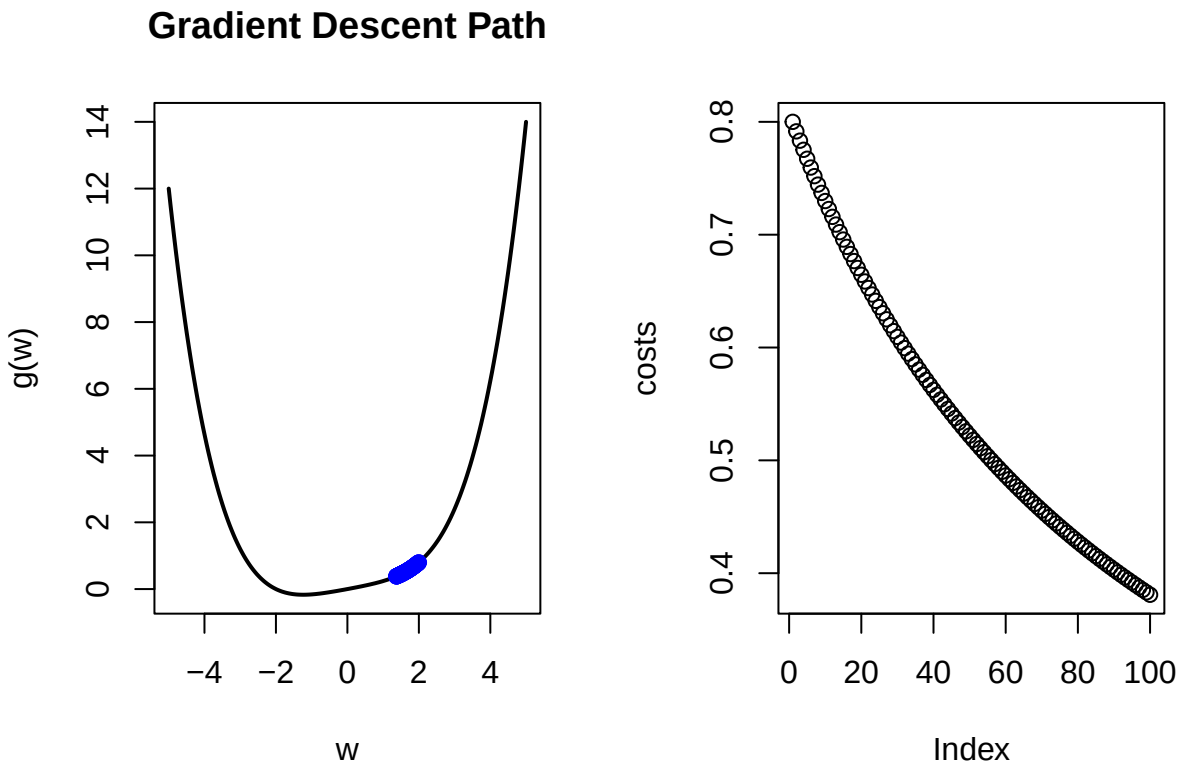

```
## [1] -0.1699693
```

```r
results <- gd(g5, gprime, alpha_choice = 0.1, w0 = 2)
plot_gd(results)
```




```
## [1] -0.1221382
```

```r
results <- gd(g5, gprime, alpha_choice = 0.01, w0 = 2)
plot_gd(results)
```

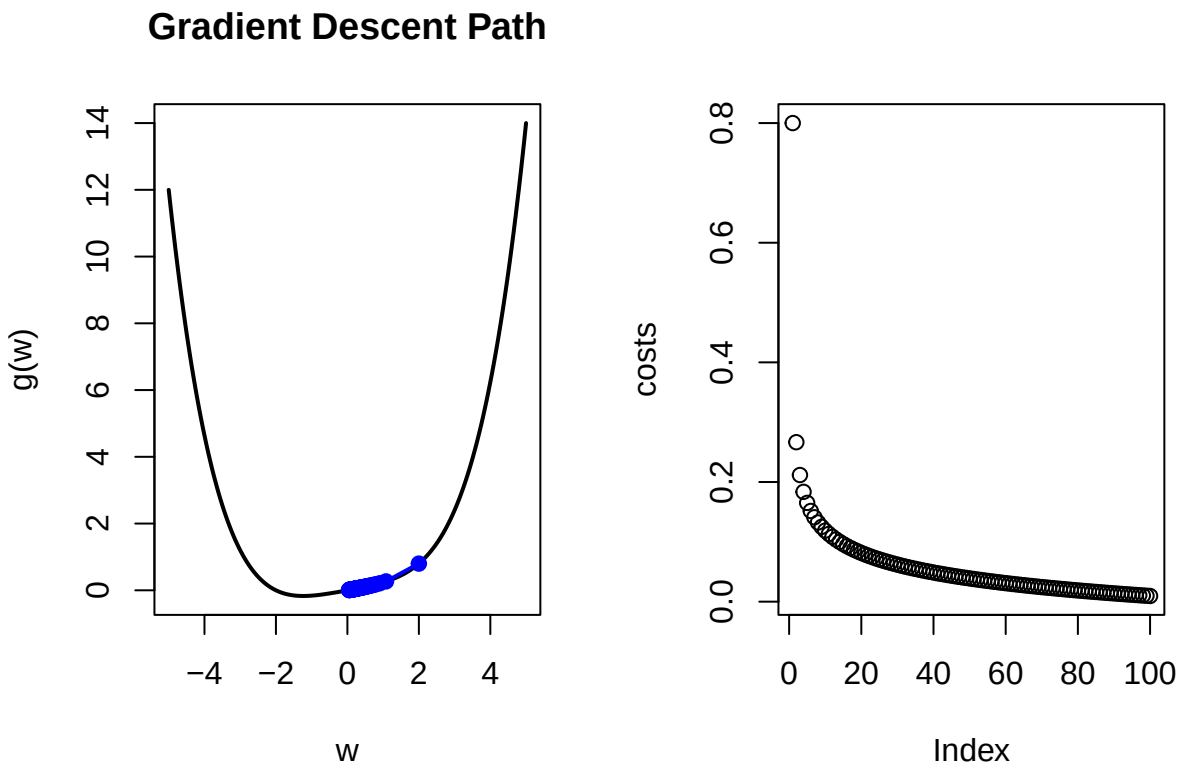



```
## [1] 0.3809034
```

From the three trials, we can see that alpha choice matters when using gradient descent. The worst result was using 0.01, which was so slow that it never found the minima. The 1 and 0.1 choices ended up close to the minima, but it took the 0.1 choice many more iterations. Given the starting location, 1 was the best step length for this gradient descent problem.

(c) Repeat (b) with diminishing steplength

```
results <- gd(g5, gprime, alpha_choice = "diminishing", w0 = 2)
plot_gd(results)
```




```
## [1] 0.009488831
```

Using the diminishing rule, we run into the same issue as when we used step size 0.01 where the function fails to find the minima before the number of iterations is reached. The initial weight of 2 is too far from the minima for the diminishing rule to be useful, so in this case we would not want to use it.

(d) Find the minima of the two functions

```r
g6 <- function(w) {
  w[1]^2 + w[2]^2 - 2*w[1] - 2*w[2] + 6
}

g6_prime <- function(w) {
  c(
    2*w[1] - 2,
    2*w[2] - 2
  )
}
```

```r
g7 <- function(w) {
  (w[1]^4 + w[2]^4)/4 - (w[1]^3 + w[2]^3)/3 - w[1]^2 - w[2]^2 + 4
}

g7_prime <- function(w) {
  c(
    w[1]^3 - w[1]^2 - 2*w[1],
    w[2]^3 - w[2]^2 - 2*w[2]
  )
}
```

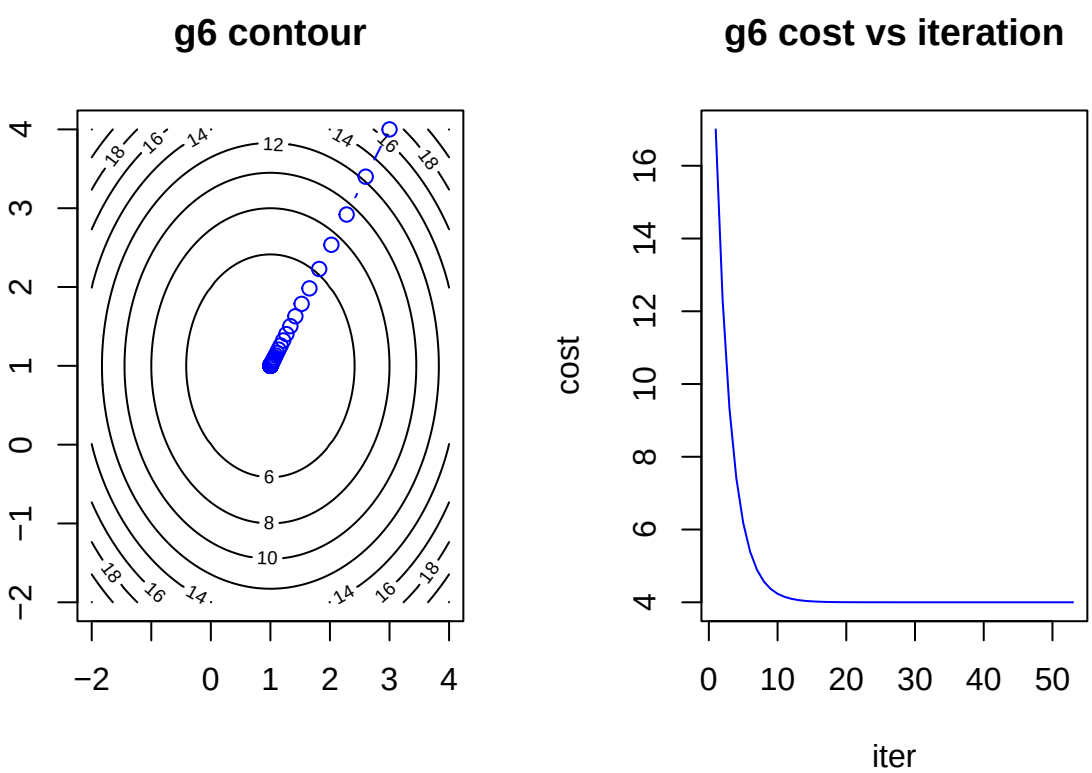
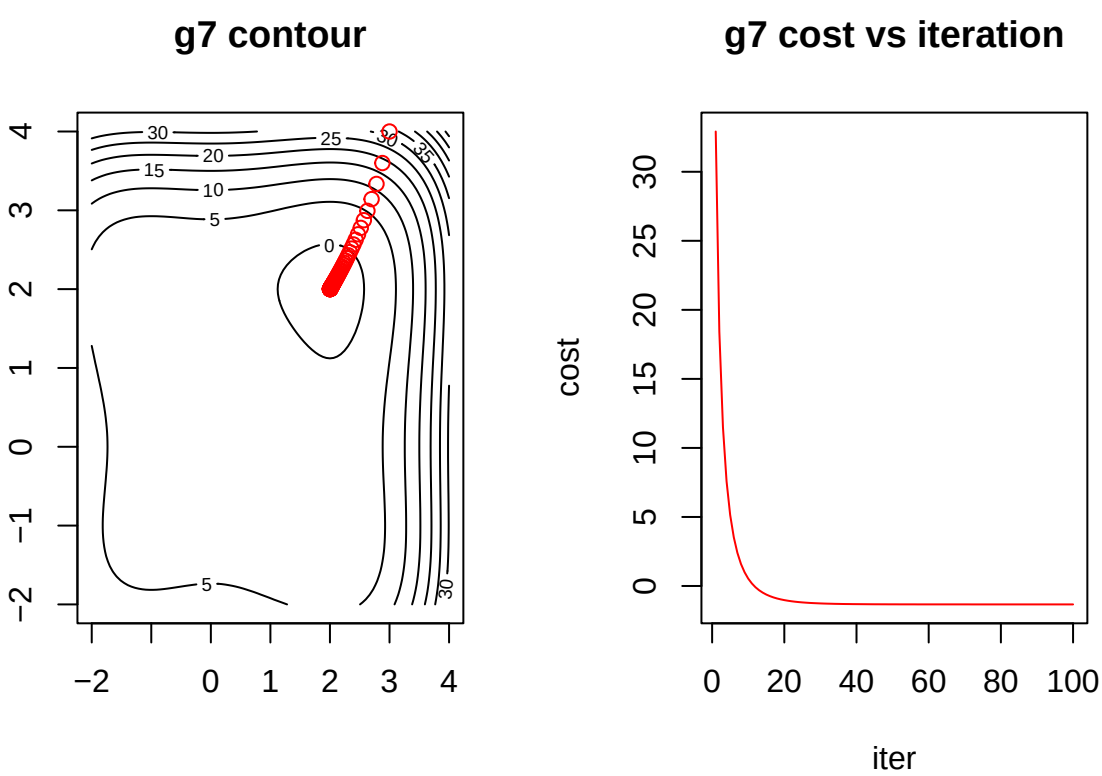
```r
w0 <- c(3, 4)

results1 <- gd(g6, g6_prime, alpha_choice = 0.1, w0 = w0)
c(Minima = tail(results1$cost_history, n = 1))
```

```
## Minima
##      4
```

```r
results2 <- gd(g7, g7_prime, alpha_choice = 0.01, w0 = w0)
c(Minima = tail(results2$cost_history, n = 1))
```

```
##    Minima
## -1.333324
```

```r
# First function plot
par(mfrow = c(1,2))
x <- y <- seq(-2, 4, length.out = 100)
z1 <- outer(x, y, Vectorize(function(a,b) g6(c(a,b))))

contour(x, y, z1, main = "g6 contour")
W1 <- results1$weight_history
points(W1[1,], W1[2,], col="blue", type="b")

plot(results1$cost_history, type="l", col="blue",
     main="g6 cost vs iteration", xlab="iter", ylab="cost")
```



```r
# Second function plot
par(mfrow = c(1,2))
z2 <- outer(x, y, Vectorize(function(a,b) g7(c(a,b))))

contour(x, y, z2, main = "g7 contour")
W2 <- results2$weight_history
points(W2[1,], W2[2,], col="red", type="b")

plot(results2$cost_history, type="l", col="red",
     main="g7 cost vs iteration", xlab="iter", ylab="cost")
```